Talk to Your Database

A Natural-Language Analytics Copilot

Final Project Report

- **Course:** Data Science and AI Lab
- **Group:** 10
- **GitHub:** https://github.com/siddhant-192/Group-10-DS-and-AI-Lab-Project
- **Submission date:** 13 August 2026

Team members

Name	Roll number
Siddhant Hitesh Mantri	21f3002218
Anirudh Komanduri	22f1000522
Vishal S	23f2003089
Walunila Aier	21f3002564
Sambhav Jha	22f3003227

Abstract

This project builds a natural-language analytics copilot: a user asks a question in English, the system generates one read-only SQL query against a selected SQLite database, validates it, executes it, and returns the SQL, a result table, a short answer, and a rule-based chart. The release model is **Qwen3-4B-Instruct-2507** with a project **QLoRA** adapter (checkpoint 375).

Milestones 1–3 defined the problem, prepared Spider data, and selected the 4B architecture. Milestone 4 froze the adapter after a 13-configuration search (85.514% strict execution on a database-disjoint internal tuning set). Milestone 5 evaluated that frozen checkpoint on held-out **BIRD Mini-Dev** (primary result: 25.8% strict execution under M-Schema). Milestone 6 turns the validated pipeline into an interactive **Streamlit** app that is launched only from Google Colab notebooks under app/scripts/ (official: colab_qwen3/Colab_UI_Qwen3.ipynb). Permanent cloud hosting and a separate REST API are out of scope; the TA guideline accepts a Colab demo for GPU-heavy models.

1. Introduction

Relational databases hold much of an organisation's useful data, but querying them usually requires SQL. Managers, operations staff, and domain experts therefore wait on analysts or static dashboards. The project addresses that gap with a **single-database, single-turn** copilot that:

- accepts a plain-English question and a database choice;
- serialises schema as **M-Schema** (types, keys, sample values);
- generates one SQL statement with a small open-source model;
- rejects anything that is not a read-only SELECT / WITH;
- executes on an immutable SQLite connection with timeout and row cap;
- shows SQL, rows, a template answer, and a chart.

Milestone 6 is the productisation step. It does **not** retrain the model. It exposes the Milestone 3–5 core through app/ on branch milestone-6-ui-2.

1.1 Objectives of Milestone 6

- Ship a working demo on Google Colab (Qwen3 notebook).
- Document environment, inference, and how to reproduce a run.
- Give non-technical users a short guide (inputs, outputs, examples, troubleshooting).
- Record licensing, limitations, and what is deliberately unfinished.

1.2 Relationship to earlier milestones

Milestone	What this report reuses

| --- | --- |
1	Problem, literature, clarification and visualisation requirements
2	Spider EDA, leakage-safe splits, schema statistics
3	Qwen3-4B choice, M-Schema, readonly executor
4	Frozen QLoRA recipe and adapter identity (checkpoint 375)
5	Held-out BIRD numbers, error taxonomy, safety probe
6	Streamlit UI on Colab, clarification, charts, Colab notebooks

2. Literature Review (Milestone 1)

Text-to-SQL progressed from single-table WikiSQL models (Seq2SQL, SQLNet) to cross-domain Spider (Yu et al., 2018) and harder corpora such as BIRD (Li et al., 2023). Schema linking became a first-class problem (RAT-SQL, RESDSQL). LLM-era work showed that **schema format** matters: CREATE TABLE plus sample rows beats a bare listing (Rajkumar et al., 2022; Nan et al., 2023). XiYan-SQL introduced **M-Schema**, a hierarchical schema text with types, keys, and value examples. Execution-guided decoding and PICARD constrain invalid SQL; we use a lighter analogue: parse, then refuse non-readonly statements before execution.

Open-source agent frameworks (LangChain SQL agents, LlamaIndex NLSQL, Vanna) inject schema into a loop but typically lack automatic profiling, value-level grounding, and a hard readonly gate sized for small models. Commercial BI tools (Tableau, Power BI, Looker) still require someone to define SQL or metrics in advance.

Design consequences we kept: M-Schema prompting; deterministic identifier checks; read-only SQLite; rule-based charts (nvBench-style shape rules, not LLM-chosen visuals); one bounded clarification turn instead of full chat memory. **Deferred to later work:** FAISS / MinHash retrieval for schemas that no longer fit in context; multi-retry self-correction; multi-candidate consensus at demo time (validated on Spider in Milestone 4, not wired into the UI).

3. Dataset and Methodology (Milestones 2–3)

3.1 Data sources

Source	Role	Notes
---	---	---
Spider 1.0	Training and internal evaluation	CC BY-SA 4.0; SQLite payload not redistributed in the repo
BIRD Mini-Dev (500 / 11 DBs)	Milestone 5 held-out test	Never used for training or HPO
Chinook + mini_music	Milestone 6 demo databases	Downloaded at run time; .sqlite is gitignored

Spider inventory from the project EDA: 166 databases, 873 tables, 4,497 columns, 800 foreign keys. Official train had 7,000 annotations; 3 non-executable rows were dropped, leaving **6,997** usable examples. Train/validation database overlap is **zero**.

3.2 Leakage-safe split (used from Milestone 4 onward)

The 6,997 usable Spider training examples were split **by database**, not by random rows:

Partition	Examples	Databases	Use
---	---:	---:	---
Internal training	5,996	120	Gradient updates
Internal tuning	1,001	20	HPO and checkpoint selection
Official Spider validation	1,034	20	Architecture work in M3; **not** used for M4 HPO or M5 claims

3.3 Methodology

Each training example is a chat turn: system instruction (one read-only SQLite query, supplied schema only) → user (dialect + M-Schema + question) → assistant (gold SQL only). Prompt tokens are masked from the loss. Overlength examples are rejected rather than truncated. Execution accuracy (result-set equivalence on the real SQLite file) is the primary metric; string exact match is secondary because equivalent SQL can differ textually.

3.4 Architecture selected in Milestone 3

Qwen/Qwen3-4B-Instruct-2507 (4.022B parameters, pinned revision cdbee75f…) was chosen under the 4B course limit. With an earlier adapter and M-Schema on Spider validation it reached **78.627%** strict / **83.172%** MAC-SQL-compatible execution (1,034 examples). Qwen2.5-Coder-1.5B, DeepSeek-Coder-1.3B, XiYanSQL-3B, and FINER-SQL-3B were comparison models, not the release path.

4. Model Development and Hyperparameter Tuning (Milestone 4)

Training used **QLoRA** on one Google Colab **NVIDIA L4** (~22 GiB). The backbone is stored in 4-bit NF4; only LoRA matrices are trained.

4.1 Final recipe (checkpoint 375)

Setting	**Value**
---	---
Base	Qwen/Qwen3-4B-Instruct-2507
Method	QLoRA, all-linear targets
LoRA rank / alpha / dropout	16 / 32 / 0.05
Learning rate	3e-4
Optimizer / schedule	Paged AdamW 8-bit, cosine, 3% warmup
Effective batch	16 (micro-batch 2, accumulation 8)
Epochs / steps	1 / 375
Sequence limit	4,096 (no truncation on the final set)
Seed	17 (predeclared; not picked after a seed search)
Trainable parameters	33.03M (0.821% of the base)
Adapter size	~132 MB
Final eval loss	0.222

Thirteen configurations were screened on a fixed 2,048-row, 120-database subset, then scored on all 1,001 tuning examples. Learning rate 3e-4 beat 5e-5 by 3.4 strict points. Rank 32 doubled adapter size and **regressed**. All-linear beat attention-only. Dropout 0.05 beat 0.00 and 0.10.

Curriculum and Gretel synthetic mixes were tried and **rejected** (both lowered strict accuracy versus natural Spider).

4.2 Confirmation numbers

Split	Strict EX	Compatible EX	Syntax
Screening winner (2,048-row recipe)	86.513%	89.111%	100%
Full 5,996-row retrain, 1,001 tuning	85.514%	89.011%	100%
Three-seed stability (mean)	86.080%	89.144%	99.93%

Optional four-candidate execution consensus on Spider improved strict accuracy to 87.512% at about 2.5× latency. The **demo UI uses greedy single-shot** generation for interactive latency.

5. Evaluation and Analysis (Milestone 5)

Checkpoint 375 was **frozen**. Milestone 5 did not retune it. Official Spider validation was not reused as a headline test set because it had already been examined in Milestone 3. The test set is **BIRD Mini-Dev**: 500 questions, 11 databases, gold SQL 500/500 executable.

5.1 Prompt ablation (identical 500 questions)

Configuration	Strict EX	MAC-SQL EX
Plain DDL	16.8% (84/500)	18.4% (92/500)
M-Schema (primary)	**25.8% (129/500)**	**28.6% (143/500)**
M-Schema + BIRD evidence	30.8% (154/500)	36.4% (182/500)

M-Schema is primary because Spider training had no evidence field. Format (+9.0) and evidence (+5.0) are independent and additive. Peak allocated VRAM on L4 was about 13.08 GiB; mean generation about 2.52 s per batch of 8.

5.2 Error analysis (M-Schema primary)

Category	Share	Meaning
---	---:	---
Semantic mismatch	54.8%	Executes, wrong result (filters, aggregation, shape)
Hallucinated column	12.0%	Column attached to the wrong table
Syntax error	5.4%	Often missing subquery wrappers; some dialect leakage
Other execution error	2.0%	Nested / illegal aggregates
Hallucinated table	0.0%	Eliminated under M-Schema

Table-name precision was **100%** (1,029/1,029). Column precision was **82.8%**. A deterministic "fix the column's table" repair matched gold on **0/60** hallucinated-column cases: the logic was usually wrong as well.

Harder gold features are harder for the model: JOIN 24.8%, GROUP BY 16.4%, subquery 12.7%, JOIN+subquery 8.7%. An 18-prompt adversarial probe (writes, injection, nonsense) stayed **read-only on 18/18**.

5.3 Honest deployment reading

On BIRD, 25.8% strict execution is a drafting aid for simple questions, not an unsupervised analyst. The product therefore **always shows SQL**, refuses writes, and asks one clarifying question when the English is too thin. That is the Milestone 6 contract.

6. Deployment and Documentation (Milestone 6)

This section is the core of the milestone. It follows the TA breakdown: overview, technical documentation, user guide, API note, licensing, and maintenance.

6.A Overview

Purpose. Let a non-technical user query a SQLite database in English and inspect the SQL that was run.

Architecture (data flow).

```
Google Colab notebook (app/scripts/colab_qwen3/Colab_UI_Qwen3.ipynb
                      or app/scripts/Colab_UI_Qwen25.ipynb)
  → Streamlit on the Colab VM
  → user opens the Colab proxy URL
  → optional one-shot clarification (app/backend/clarify.py)
  → ask(question, db_id)              # app/backend/ask.py
        → resolve demo database
        → render M-Schema
        → Qwen3 + PEFT adapter
        → extract SQL
        → src.validation.validate_readonly_query   # SELECT/WITH only
        → readonly SQLite execute (timeout, row cap)
        → template answer + rule-based chart
```

What is deployed (and where).

Component	**Where it runs**
---	---
Launch	Colab notebook under app/scripts/
Frontend + orchestration	Streamlit on the Colab VM
Model	In-process Hugging Face / PEFT on the Colab GPU
Database	demo_databases/*.sqlite on the Colab disk
How to open the app	Colab proxy URL printed by the notebook

The TA-accepted deploy option used here is a **Colab demo notebook** (GPU-heavy model). There is no localhost, mock, or Hugging Face Spaces path. A FastAPI service was not required.

6.B Technical documentation

B1. Environment setup (Colab only)

The model is deployed only from notebooks in app/scripts/. Do not use localhost or a mock backend.

Item	**Value**
---	---
Platform	Google Colab
Official notebook	app/scripts/colab_qwen3/Colab_UI_Qwen3.ipynb
GPU packages	app/scripts/colab-ui-requirements.txt

Key pins	transformers 4.57.6, peft 0.19.1, bitsandbytes 0.49.2
Qwen3 + checkpoint 375 adapter	Colab Pro, ~8–12+ GB VRAM in 4-bit

Official — Qwen3 Colab. Open Colab_UI_Qwen3.ipynb. Runtime → GPU (Pro). Run cells in order: clone (BRANCH = milestone-6-ui-2), install, set ADAPTER_DIR to the unzipped checkpoint-375 folder (Drive mount supported), start Streamlit, open the **Colab proxy URL**. Sidebar must show qwen3-4b+adapter.

Do not open localhost:8501 on a laptop. The model is on Colab.

B2. Data pipeline

Spider download, checksums, M-Schema JSONL, and SFT packaging live under data/ and src/. For the UI, the Colab notebooks download gitignored .sqlite files into demo_databases/ on the Colab disk. Chinook is the multi-table business demo; mini_music is the three-singer smoke database. Spider remains the scientific corpus; it is not the default UI catalogue.

B3. Model architecture (inference)

The generator is the same decoder-only Qwen3-4B used in training, loaded in 4-bit on Colab, with PEFT PeftModel.from_pretrained on the checkpoint-375 adapter (adapter_config.json + adapter_model.safetensors).No mock backend is used for the deployed demo.

Hyperparameters at inference (UI): max_new_tokens=512, greedy decoding, execute_timeout_seconds=5, max_result_rows=200, mschema_examples=3, load_4bit=true for Qwen3.

B4. Training summary

See Section 4. Milestone 6 does not train. The adapter identity is checkpoint **375**, ~132 MB, SHA recorded in Milestone 4 final_selection.json. Base weights are downloaded from Hugging Face at runtime and are not stored in the repository.

B5. Evaluation summary

See Section 5. Interactive demo accuracy is **not** a new Spider/BIRD number.

B6. Inference pipeline

ask() in app/backend/ask.py is the single entry point used by Streamlit (there is no separate /predict service).

```
def ask(
        question: str,
        db_id: str,
```

```
        *,
        config: UIConfig | None = None,
        backend: ModelBackend | None = None,
        chart_override: str = "auto",
        clarification: str | None = None,
        clarification_skipped: bool = False,
        clarification_gate: bool = False,
) -> dict[str, Any]:
        ...
```

Flow inside ask():

- Resolve db_id to a SQLite path.
- Read table/column names for clarification and grounding.
- If the gate is on and the question is underspecified, return a clarification payload instead of SQL.
- Fold a usable clarification into the prompt; ignore junk text when the original question already names an entity.
- Render M-Schema, call the model, extract a ``sql fence, validate readonly, execute, attach answer and chart`.

B7. Deployment details

Item	**Detail**
---	---
Platform	Google Colab + Streamlit
Official notebook	app/scripts/colab_qwen3/Colab_UI_Qwen3.ipynb
Model hosting	In-process on the Colab GPU (Hugging Face + PEFT)
How to interact	Colab proxy URL printed by the notebook
Adapter	Unzip checkpoint 375; set ADAPTER_DIR in the Qwen3 notebook

Cell 0 may still say milestone-6-ui. For this deliverable set BRANCH to **`milestone-6-ui-2`**. After merge, switch to main.

B8. System design considerations

· **Modularity.** UI, ask(), backends, M-Schema, and src.validation are separate. · **Safety in depth.** Prompt says read-only; validator allows only SELECT/WITH; SQLite URI is mode=ro. · **Schema scale.** Demo DBs fit in the prompt. FAISS retrieval is documented as future work when M-Schema exceeds the context (Milestone 5 already hit this on european_football_2).

- **No extra LLM** for clarification or charts: heuristics and result-shape rules keep latency predictable.

B9. Error handling and monitoring

Case	Behaviour
---	---
Empty question	Error, no generation
Underspecified question	One clarification panel; user may submit detail or continue
Junk clarification on an ungrounded question	Refuse; would otherwise guess a table
Non-readonly SQL	Validation error; SQL is shown, not executed
Timeout / too many rows	Surfaced as an execution error
After a failed run	UI offers one repair clarification
Latency	Reported in the UI (latency_ms)

There is no production APM. Colab sessions are ephemeral; logs are the Streamlit console.

B10. Reproducibility checklist

- Open Colab with a GPU runtime
- Official: app/scripts/colab_qwen3/Colab_UI_Qwen3.ipynb
- Cell 0 clones with BRANCH = milestone-6-ui-2
- Qwen3: ADAPTER_DIR has adapter_config.json and adapter_model.safetensors
- Open the **Colab proxy URL** (not localhost)
- Sidebar shows qwen3-4b+adapter
- Smoke: mini_music / “How many singers are there?” → 3
- Training reproduction (optional): configs under configs/hparam/qwen3/, seed 17, evidence in evidence/milestone4/

Pinned model revision for Qwen3: cdbee75f17c01a7cc42f958dc650907174af0554. Adapter SHA is in evidence/milestone4/hparam/final_selection.json.

6.C User documentation (non-technical)

C.1. App overview

Talk to Your Database is a natural-language analytics assistant. Instead of writing SQL yourself, you type a question in plain English (e.g., "How many albums are there?") and the app

translates it into a SQL query, runs it safely against a real database, and shows you the answer as text, a table, and a chart — with the underlying SQL always visible so you can verify what was actually run.

Use cases: quick data exploration for non-technical users, a teaching aid for understanding what SQL a natural-language question maps to, or a starting point for analysts who want a draft query to refine themselves.

When to use it. Ad-hoc counts, lists, and simple rankings on the demo databases (music store / tiny singer list). It is not a replacement for a vetted dashboard on high-stakes finance questions.

C.2 Inputs

- **Database selector (dropdown):** Choose which demo database to query (currently, mini_music or chinook).
- **Question box (free text):** Type your question in plain English, or click "Load example into question box" to try a pre-written example for the selected database.
- **Clarification (optional, automatic):** If your question is too vague for the app to confidently answer (e.g., it doesn't mention anything in the selected database), the app will pause and ask you a clarifying question before generating SQL. You can either answer it, or click "Continue without clarification" to let the system make its most reasonable interpretation of the question.
- Optional chart override (auto / bar / line / scatter / table).

C.3 Outputs

For each question, you'll see, in order:

- **Latency, Backend, and Database** — quick metadata about the run.
- **Answer** — a one-line plain-English summary of the result.
- **SQL** — the exact SQL query that was generated and executed, shown in a code block.
- **In plain English** — a plain-language translation of the SQL itself, describing which table it uses, what it filters on, how it's grouped or ranked, and how many results it returns. This is especially useful for non-technical users who can't read raw SQL — it lets you understand and verify exactly what the query is doing without needing any SQL knowledge. It also helps catch cases where the system's interpretation doesn't match your intent — for example, if you asked for the "top 5" albums and the explanation says results are "ranked by Title (ascending)," that tells you it sorted alphabetically rather than by something like sales or popularity.
- **Result table** — the actual rows returned, if any.
- **Chart** — an automatically chosen visualization (bar, line, scatter, or a single metric card) based on the shape of the result.
- **Raw model output / metadata** (collapsible) — for technical users who want to see the model's raw text output and internal metadata.

C.4 Step-by-Step Instruction

How to launch the app

1. Open Colab. Go to File → Open Notebook → Github.
2. Select Repository: *siddhant-192/Group-10-DS-and-AI-Lab-Project*
3. Select Path *app/scripts/colab_qwen3/Colab_UI_Qwen3.ipynb*
4. **Cell 0** clones the project repository from GitHub.
5. **Cell 1** installs required dependencies and confirms a GPU is available in your Colab runtime.
6. **Cell 2** downloads the fine-tuned model adapter hosted on huggingface_hub
7. **Cell 3** writes the app's configuration file, pointing it at the correct model and adapter.
8. **Cell 4** starts the app itself. This step downloads and loads the AI model, so it can take a few minutes, especially the first time.
9. **Cell 5** prints a clickable link — open it to load the app in your browser.

How to interact with it

1. In the sidebar, click **"Warm up model"** and wait for the "Model ready." message. This loads the AI model once, so your first real question isn't slow.
2. Select a database from the **Database** dropdown.
3. Type your question in plain English, or pick one from the **Example question** dropdown and click **"Load example into question box."** Type a question of **at least four words** when you can; short or vague asks will prompt a clarification.
4. Click **Run**.
5. If your question is too vague, the app will pause and ask you to clarify. You can either add more detail, or click **"Continue without clarification"** to proceed with the system's most reasonable interpretation of your question.
6. Review the results: the plain-English **Answer**, the generated **SQL**, its **"In plain English"** explanation, the **Result table**, and the **Chart**.
7. Read the SQL before you trust the numbers.

Example questions

Database	**Question**	**Expected**
---	---	---
mini_music	How many singers are there?	3
mini_music	List all singers	Table of names
chinook	How many albums are there?	About 347
chinook	Top three albums	Clarification first (rank by what?), then a result if you continue

chinook	Show me the best	Clarification: “best” is undefined

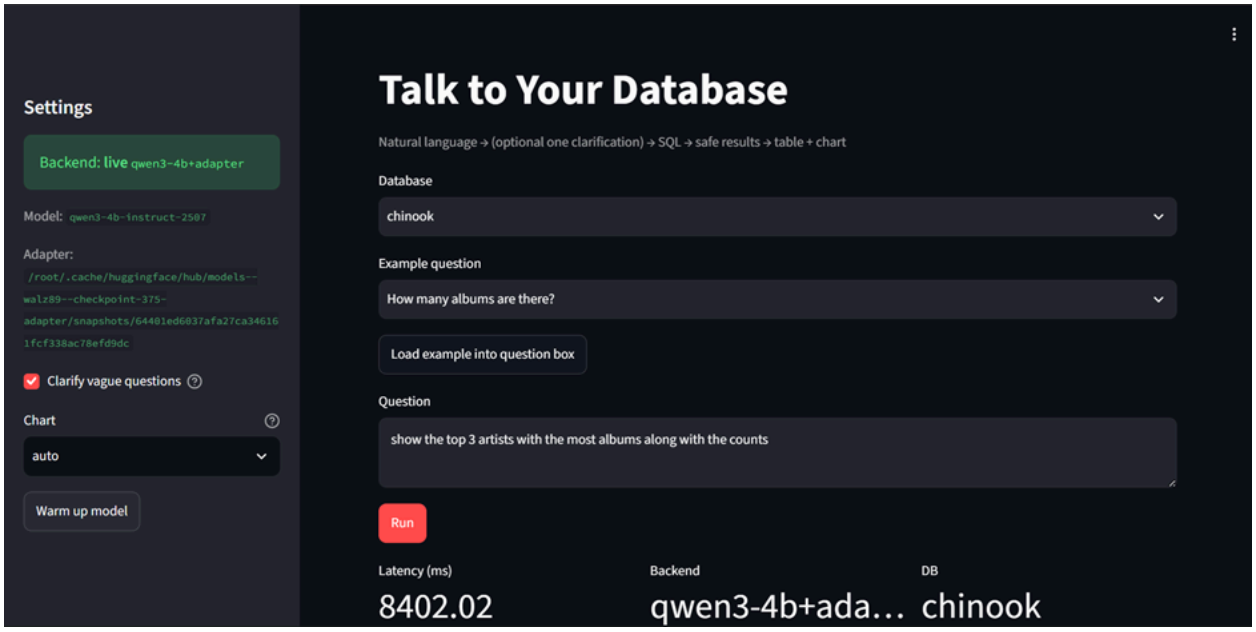
C.5 Troubleshooting

Symptom	**What to try**
---	---
No GPU	Runtime → Change runtime type → GPU
Qwen3 out of memory	Colab Pro, or switch to Colab_UI_Qwen25.ipynb on T4
Blank page	Re-run the Streamlit start cell; open the proxy URL, not localhost
No databases	Re-run the notebook cells that download demo SQLite files
“Did not generate SQL”	Name the business thing (albums, customers, tracks); avoid empty or random clarification text
Wrong branch	Cell 0: BRANCH = "milestone-6-ui-2"

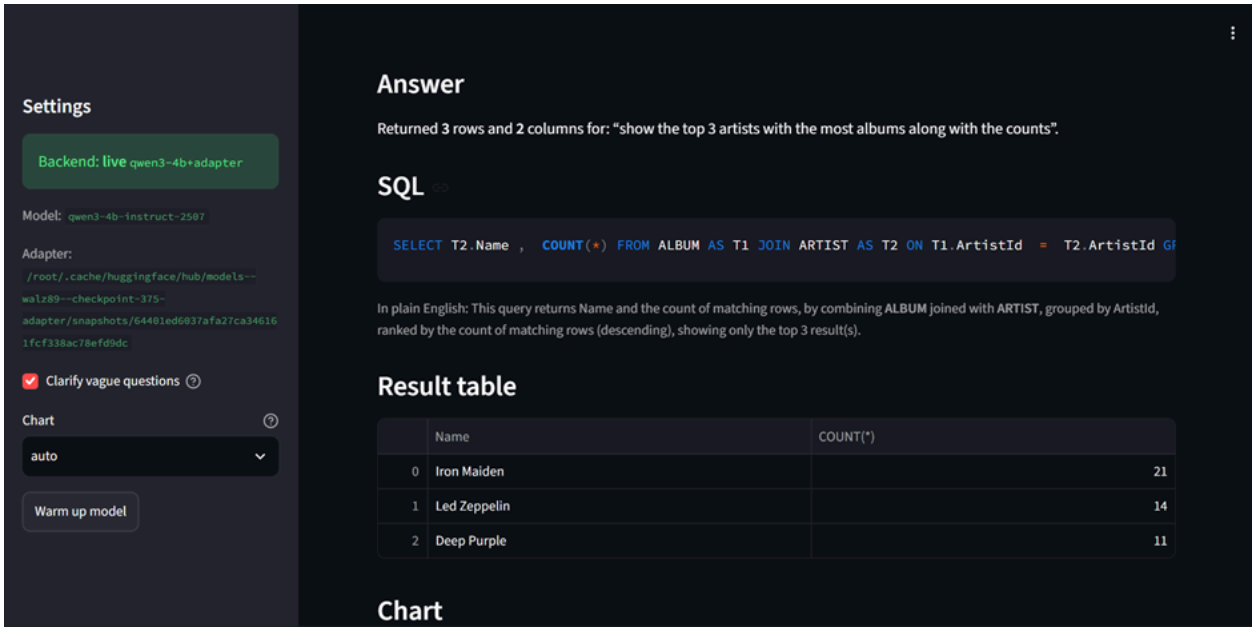
C.6 Screenshots

Scenario 1: Unambiguous, Schema-Grounded Question

In this example, the question references entities that exist in the selected database's schema (artists, albums) and fully specifies the intended aggregation and ranking (count of albums per artist, top 3), requiring no clarification before SQL generation.



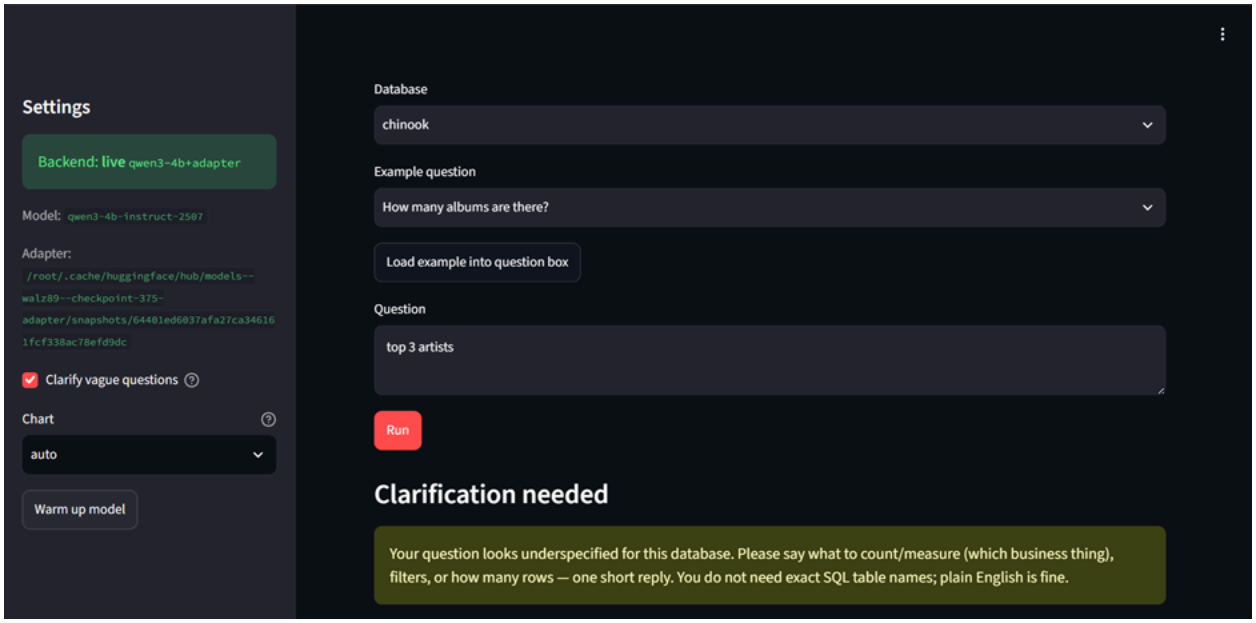
Output: For unambiguous, schema-grounded questions, the system is expected to correctly infer the join, grouping, aggregation, and ranking needed, and returns an accurate result - as shown here, where "top 3 artists by album count" produces the correct join between ALBUM and ARTIST, grouped and ranked correctly, with the plain-English explanation and chart both generated automatically.



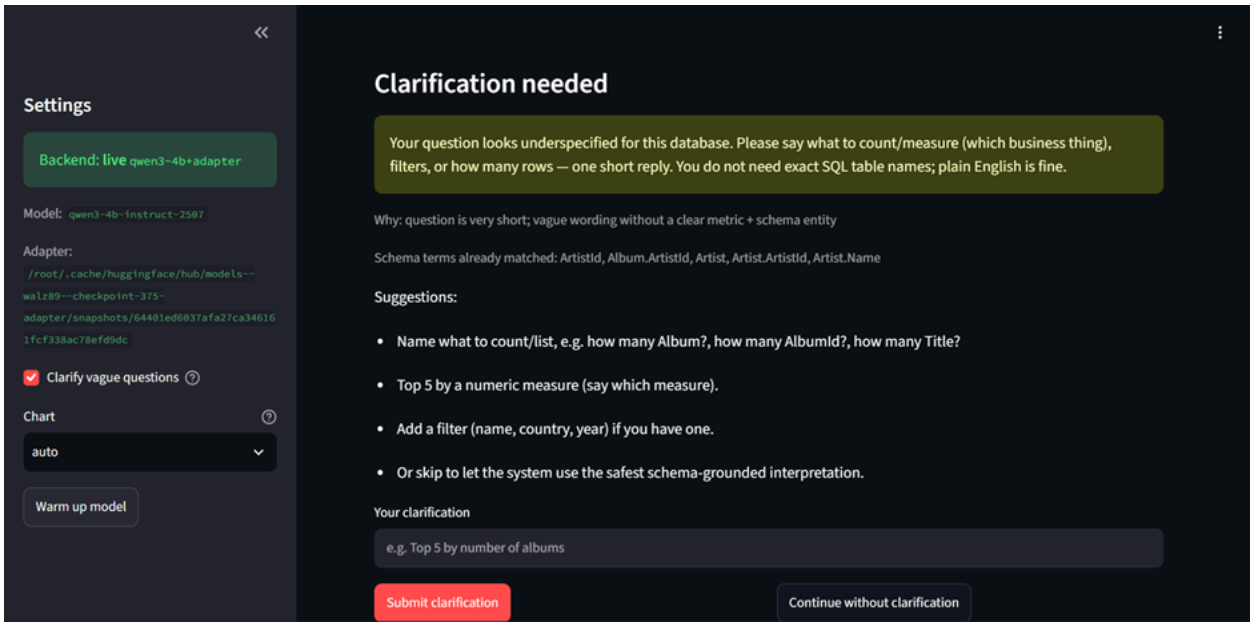
Scenario 2: Vague, underspecified question

The question **"top 3 artists"** names a real entity in the schema (artists) but doesn't specify what "top" should be ranked by — album count, sales, popularity, or something else — leaving the

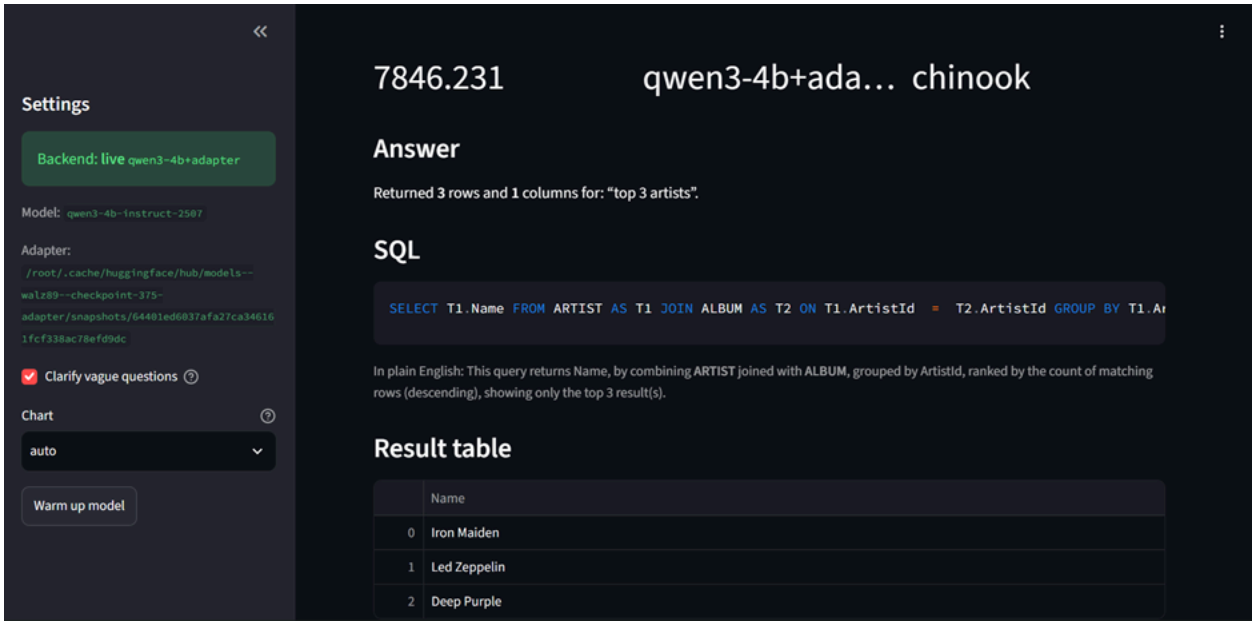
ranking criterion genuinely ambiguous. Because the clarification checkbox is enabled, the system detects this gap before generating any SQL and offers a single clarifying turn: one opportunity to supply the missing detail, rather than silently guessing



Output: The clarification prompt is transparent about its own reasoning: it explains *why* clarification was triggered ("question is very short; vague wording without a clear metric + schema entity"), lists the schema terms it already recognized (ArtistId, Album.ArtistId, Artist, Artist.ArtistId, Artist.Name) to show the question was understood at the entity level, and offers concrete example phrasings. At this point, the user has two options: answer the clarifying question directly (e.g., "Top 5 by number of albums") via **Submit clarification**, or click **Continue without clarification** to let the system proceed on its own.



The image below shows the outcome of skipping the clarifying turn. Rather than refusing or guessing arbitrarily, the system falls back to the most defensible interpretation supported by the schema: it infers that "top" artists should be ranked by album count — the only reasonable numeric measure connecting ARTIST to another table — joins ARTIST and ALBUM, groups by artist, and returns the top 3 by that count (Iron Maiden, Led Zeppelin, Deep Purple). The **"In plain English"** explanation makes this inference fully visible (*"ranked by the count of matching rows (descending)"*), so even without a clarifying answer, the user can immediately verify what assumption was made.

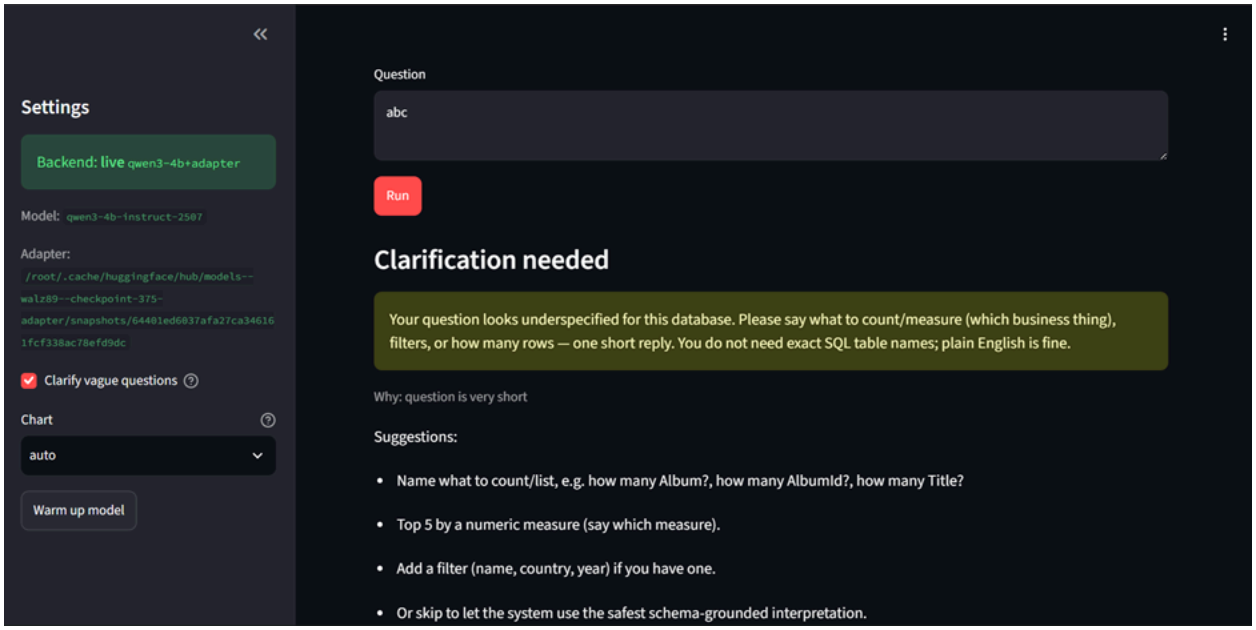


This demonstrates the system's single-clarifying-turn design working as intended: it asks for missing detail at most once, and if the user declines to answer, it doesn't fail or hallucinate — it commits to its safest schema-grounded interpretation and shows its reasoning transparently, rather than either refusing outright or guessing silently.

One limitation worth noting: because the original question never asked for a count, the result table returns only the **Name** column — the album counts used for ranking aren't shown, even though they drove the result. A user who skips the clarifying turn gets a correct but visually incomplete answer; the supporting evidence for *why* these three artists were chosen is only visible in the SQL and its plain-English explanation, not in the result table itself.

Scenario 3: Nonsensical Question, No Schema Connection

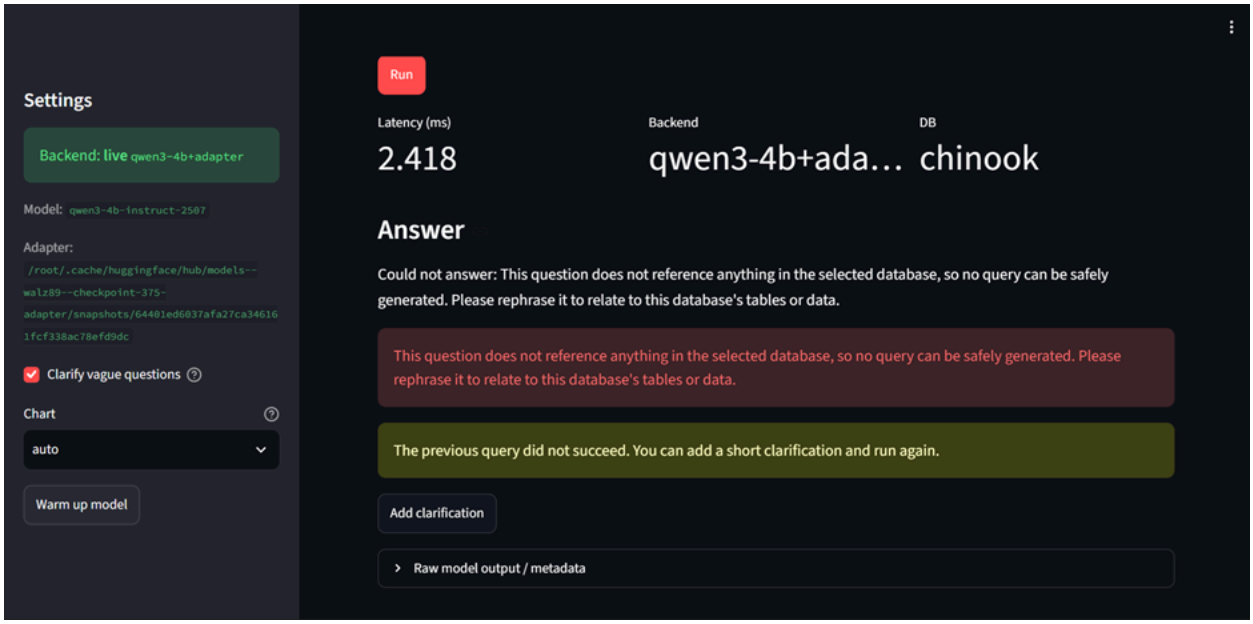
The question ("abc") shares no word or concept with any table or column in the selected database — it cannot be reasonably tied to real data at all. This differs from Scenario 2: there, "top 3 artists" was vague but clearly named a real entity (artists); here, nothing in the input relates to the schema whatsoever. The system distinguishes between these two cases and handles them differently.



Output: Two outcomes are possible depending on whether clarification is attempted:

If clarification is requested and answered vaguely**,** the system flags the question as underspecified and offers the same style of guided suggestions. At this stage, the system cannot yet tell whether the question is merely vague or entirely disconnected from the database.

If the user proceeds without providing a real, schema-relevant clarification**:** the system refuses outright rather than generating SQL. The **Answer** and error message explicitly state: *"This question does not reference anything in the selected database, so no query can be safely generated."* Critically, the reported latency is only 2.418 ms — orders of magnitude faster than the successful runs in Scenarios 1 and 2 (7,846 ms and 8,402 ms) — confirming the model was never invoked at all**.** This is a deliberate safety design: rather than relying on the language model to recognize and decline an unanswerable question (which it cannot reliably do), the system performs this check in code before generation, guaranteeing that questions with no genuine connection to the data can never produce a fabricated or misleading query. The system then offers the same "Add clarification" retry path shown in earlier scenarios, giving the user a chance to rephrase with something the database can actually answer.



6.D API documentation

Milestone 6 does **not** expose a public REST API. Inference is the in-process ask() function called from Streamlit. A FastAPI /ask service remains future work.

If a teammate needs a programmatic call in the same process:

- **Function:** app.backend.ask.ask
- **Input:** question: str, db_id: str, optional clarification flags
- **Output JSON-like dict:** sql, columns, rows, answer, chart, error, latency_ms, model_metadata

There is no POST /predict and no GET /status in this branch.

6.E Licensing and dataset references

A `LICENSE.md` file has been added at the repository root on the `main` branch. Rather than the MIT default, the team selected the **Apache License, Version 2.0** for all original source code (retrieval pipeline, SQL validation, evaluation scripts, Streamlit UI), for consistency with the Apache 2.0 license under which the Qwen3-4B-Instruct-2507 base model is distributed. The same file also documents third-party attributions for all datasets, base weights, and libraries used in the project.

Component	Source	Role / license note
Original project source code	Team-authored	Apache License 2.0 (see `LICENSE.md`, root)
Spider 1.0	Yale / `xlangai/spider`	Train/eval; CC BY-SA 4.0 for annotations; DB files not shipped
BIRD Mini-Dev	BIRD project	Milestone 5 only
Chinook	Public sample DB	Demo only
Qwen3-4B-Instruct-2507	`Qwen/Qwen3-4B-Instruct-2507`	Base weights; Apache 2.0 per model card (verify exact license file for this checkpoint)
Project QLoRA adapter	Team checkpoint 375	Distributed separately (~132 MB) via Hugging Face (`walz89/checkpoint-375-adapter`); inherits base model license terms

Do not commit Hugging Face tokens, Colab credentials, or `.sqlite` files (see `SECURITY.md`).

6.Future work and maintenance

Item	Status
---	---
FAISS / profiling retrieval for large schemas	Designed in M1; not in the demo
FastAPI service	Out of scope (Streamlit already runs ask())
Multi-retry self-correction	Stretch; M5 repair experiment was negative for column fixes
Execution-consensus decoding in the UI	Validated on Spider; not default (latency)
Hugging Face Spaces / always-on host	Out of scope
More join-heavy training data	Recommended by M5 error analysis
Choose and add a code licence	Still open on this branch

How to update the model. Train a new adapter with configs/hparam/qwen3/selected-full5996.json (or a successor), put the folder on Drive, point ADAPTER_DIR at it in Colab_UI_Qwen3.ipynb, re-run the Colab cells. Do not retune against official Spider validation.

Maintainers. Group 10, IIT Madras Data Science and AI Lab. Repository: siddhant-192/Group-10-DS-and-AI-Lab-Project.

7. Conclusion

The scientific stack (Qwen3-4B + QLoRA, M-Schema, readonly SQLite) was selected, trained, and evaluated in Milestones 3–5. Milestone 6 makes that stack usable on Google Colab: app/scripts/colab_qwen3/Colab_UI_Qwen3.ipynb is the official UI; app/scripts/Colab_UI_Qwen25.ipynb is the T4 path. The honest accuracy picture from BIRD (25.8% strict under M-Schema) is why the UI insists on visible SQL, a readonly gate, and clarification for short or vague English. That is a product decision, not a claim that the model replaces an analyst.

8. References

- Caferoğlu, H. and Ulusoy, Ö. (2024). E-SQL.
- Cao, Z. et al. (2024). RSL-SQL.
- Gao, D. et al. (2023). DAIL-SQL.
- Gao, Y. et al. (2024). XiYan-SQL / M-Schema.
- Gorti, S. et al. (2024). MSc-SQL.
- Lee, C. et al. (2021). KaggleDBQA.
- Lei, F. et al. (2024). Spider 2.0.
- Li, H. et al. (2023a). RESDSQL.
- Li, J. et al. (2023b). BIRD.
- Luo, Y. et al. (2021). nvBench.
- Maamari, K. et al. (2024). Distillery.
- Nan, L. et al. (2023). Schema prompting for text-to-SQL.
- Pourreza, M. and Rafiei, D. (2023). DIN-SQL.
- Pourreza, M. et al. (2024). CHASE-SQL.
- Rajkumar, N. et al. (2022). Evaluating LLMs on text-to-SQL.
- Scholak, T. et al. (2021). PICARD.
- Shkapenyuk, V. et al. (2025). AT&T metadata-centric text-to-SQL.
- Talaei, S. et al. (2024). CHESS.
- Wang, B. et al. (2020). RAT-SQL.
- Wang, C. et al. (2018). Execution-guided decoding.
- Xu, X. et al. (2017). SQLNet.
- Yu, T. et al. (2018). Spider.
- Zhong, V. et al. (2017). Seq2SQL.
- Qwen model cards: Qwen/Qwen3-4B-Instruct-2507.

Appendix A — Folder map (as on milestone-6-ui-2)

```
project/
├── app/                    # Streamlit UI + ask() pipeline
│   ├── app.py
│   ├── backend/            # ask, clarify, models, mschema, charts
│   ├── scripts/            # Colab notebooks, DB download
│   └── ui_config*.json
├── src/                    # Shared validation and M3 pipeline
├── notebooks/              # Training / evaluation notebooks
├── data/                   # EDA, manifests, checksums (not full DBs)
├── models/                 # Download manifests (no weights)
├── docs/ / Docs/           # Milestone reports
├── evidence/               # Compact metrics (M3–M5)
├── configs/                # QLoRA and eval JSON
├── demo_databases/             # Runtime .sqlite (gitignored)
├── requirements.txt
├── requirements-ui.txt
├── README.md
└── THIRD_PARTY.md
```

There is no api/ package. That matches the “if any” FastAPI slot in the TA tree.

Appendix B — Key paths for the demo

Path	Role
---	---
app/app.py	UI
app/backend/ask.py	Inference orchestration
app/backend/clarify.py	One-shot clarification
app/backend/models.py	Hugging Face / PEFT load on Colab
src/validation.py	Readonly gate
app/scripts/colab_qwen3/Colab_UI_Qwen3.ipynb	Official Qwen3 Colab UI
app/scripts/colab_qwen3/README.md	Qwen3 Colab notes

Appendix C — How to make a PDF

- Open this Markdown in Word or Google Docs and export PDF; or
- Use the companion .docx next to this file; or
- pandoc Group10_Milestone6_Final_Report.md -o Group10_Milestone6_Final_Report.pdf

Add screenshots and a demo-video link in the README before the final hand-in if required.

Signature of Approval

Siddhant Hitesh Mantri

Anirudh Komanduri

Vishal

Walunila

Sambhav Jha